

Notebook Construction and Audit Strategies

This section outlines two complementary frameworks for working with computational notebooks in machine learning and data science contexts: a **Construction Framework** for authoring new notebooks, and an **Audit Framework** for reviewing existing ones. Keeping these separate prevents the common mistake of evaluating while still constructing, which leads to incomplete analysis of both.

The two frameworks operate within a feedback loop, illustrated in Figure 1.

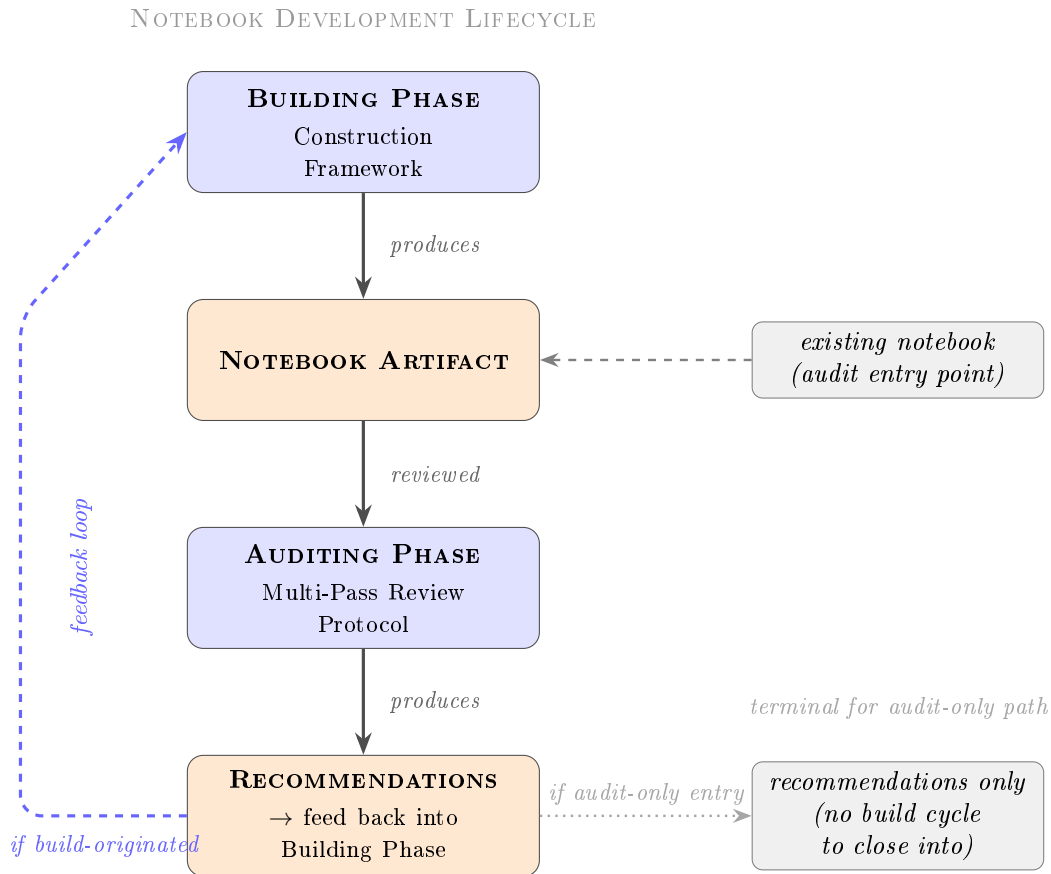


Figure 1: The Notebook Development Lifecycle. The **Construction Framework** (Part I) produces a notebook artifact, assessed by the **Audit Framework** (Part II) via a six-pass review protocol. For notebooks that originate in the Building Phase, the resulting recommendations feed back into a revised construction cycle, forming a closed feedback loop. An existing notebook (dashed entry) may instead enter the lifecycle directly at the artifact stage for audit-only workflows; in that case recommendations are a terminal deliverable rather than a loop closure, unless the notebook is subsequently routed into the Building Phase.

Part I — Notebook Construction Framework

The Construction Framework is a *forward-looking prescription*: it guides the notebook author in producing a well-structured, reproducible, and maintainable artifact from the ground up. It is organized into three sequential phases.

Phase 1 — Scaffold: Establish the structural and reproducibility foundation before writing any logic:

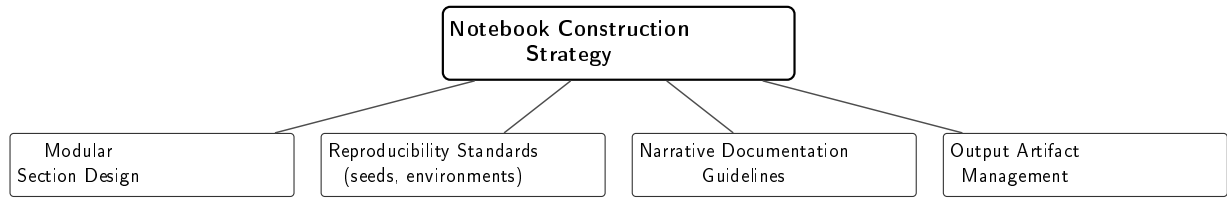


Figure 2: High-level pillars of the Notebook Construction Framework.

- Define the notebook’s **single responsibility** — one notebook should correspond to one coherent workflow or experiment.
- Create standard section headers as markdown cells, following this canonical order:
 1. Environment & Dependencies
 2. Configuration & Global Parameters
 3. Data Ingestion
 4. Preprocessing & Feature Engineering
 5. Model Definition & Training
 6. Evaluation & Metrics
 7. Artifact Export (models, plots, reports)
 8. Conclusions & Next Steps
- Pin the execution environment upfront via `requirements.txt`, `conda.yaml`, or `pyproject.toml`.
- Set global reproducibility controls: random seeds, deterministic flags, and device configuration.

Phase 2 — Write: Author each section incrementally, following these discipline rules:

- Work through **one section at a time** (Divide and Conquer principle).
- Accompany every code cell with a markdown explanation of intent and expected output.
- Avoid burying logic inside loops or helper calls without commentary.
- Prefer **explicit variable passing** over hidden or global state to maintain cell independence.
- Summarize repetitive code patterns with a general description and highlight any meaningful variations; three or more similar blocks should be treated as a refactoring candidate.
- Route all persisted outputs (models, plots, reports) through a single, versioned export convention defined at the start of the section — never write artifacts ad hoc from arbitrary cells.

Phase 3 — Validate During Writing: Continuously verify correctness as each section is completed:

- Restart the kernel and run all cells after completing each section.
- Confirm **cell idempotency**: every cell should produce the same result when re-executed independently.

- Verify that outputs match expectations before proceeding to the next section.
- Ensure cell execution order is strictly linear and free of hidden dependencies.
- Confirm that exported artifacts land in the designated output location with the expected naming and versioning convention before moving to the next section.

Part II — Notebook Audit Framework

The Audit Framework is a *backward-looking diagnostic*: it guides a reviewer, collaborator, or AI assistant in systematically assessing the quality, correctness, and risk profile of an existing notebook.

To execute this efficiently, the reviewer should employ guiding tactics such as dividing the notebook into logical modules, prioritizing critical ML pipeline blocks over repetitive exploration cells, using hierarchical descriptions (high-level summaries first, drill-downs later), and summarizing repetitive code patterns rather than analyzing each individually. When a human collaborator specifies focus areas in advance (e.g., "check only for data leakage" or "skip code style"), the reviewer should scope the six-pass review protocol to those areas rather than executing all six passes uniformly.

The framework is formalized as a six-pass review protocol, applied sequentially. Any pass included in scope is executed in full, following its own deliverable requirements below.

Where a pass deliverable is a score, the reviewer rates on a three-level scale — *Low Risk*, *Moderate Risk*, *High Risk* — based on the number and severity of unresolved items identified within that pass, rather than a numeric or percentage scale.

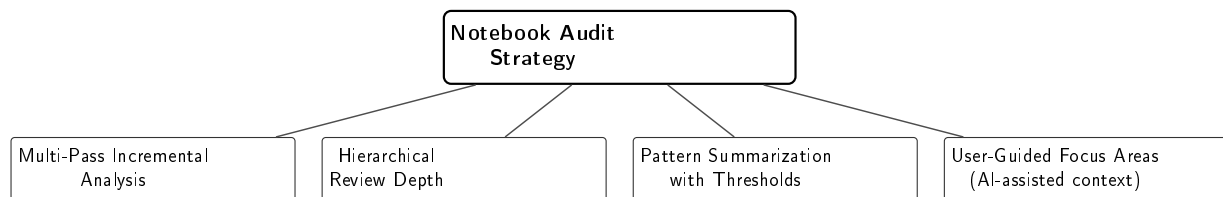


Figure 3: High-level pillars of the Notebook Audit Framework.

Pass 1 — Structural Overview: Establish a high-level map of the notebook before examining any logic:

- Record any user-specified focus areas or exclusions that will scope the remaining five passes.
- Assess whether the notebook has clear, navigable section headers.
- Confirm that the notebook has a single, coherent purpose.
- Verify that cell execution order is linear and safe.
- Identify any immediately visible red flags (e.g., missing outputs, broken imports, orphaned cells).
- *Deliverable:* Section map, preliminary red flags list, and recorded focus-area scope (if specified).

Pass 2 — Reproducibility Check: Assess whether the notebook can be reliably re-executed:

- Verify that dependencies are declared and version-pinned.
- Confirm that random seeds are set globally and consistently.

- Flag any hardcoded file paths, credentials, or environment-specific assumptions.
- Determine whether the notebook can run end-to-end on a fresh kernel without manual intervention.
- *Deliverable:* Reproducibility risk score (Low / Moderate / High).

Pass 3 — Data Integrity Review: Examine the data pipeline for correctness and leakage risks:

- Confirm that the train/test split is performed **before** any preprocessing step.
- Identify signs of **data leakage** (e.g., scaling fitted on the full dataset).
- Verify that missing data is handled consistently across splits.
- Check that data types, shapes, and distributions are validated at ingestion.
- *Deliverable:* Data pipeline integrity report.

Pass 4 — ML Correctness Audit: Evaluate the soundness of the machine learning methodology:

- Confirm that the evaluation metric is appropriate for the task and class distribution.
- Verify that cross-validation is applied correctly and without leakage.
- Ensure hyperparameter tuning is performed on validation data only, never on test data.
- Check that model performance is contextualized against a meaningful baseline.
- *Deliverable:* ML correctness checklist.

Pass 5 — Code Quality Review: Assess the maintainability and clarity of the code:

- Flag repetitive code blocks that exceed the three-block threshold as refactoring candidates.
- Identify dead code, unused imports, and redundant variable assignments.
- Assess whether variable and function names are meaningful and consistent.
- Check for bloated cell outputs (e.g., printing entire dataframes or raw tensors).
- *Deliverable:* Code quality and smell report.

Pass 6 — Deployment Readiness: Determine whether the notebook is suitable for production or publication:

- Verify that model artifacts are explicitly saved with versioned filenames.
- Confirm that inference logic is separable from training logic.
- Check that compute and memory constraints are documented or profiled.
- Ensure a complete environment export is present and up to date.
- Flag any data privacy or PII handling concerns.
- *Deliverable:* Deployment readiness score (Low / Moderate / High).

Applying the Two Frameworks

The appropriate framework depends on the context of engagement with the notebook:

Use the Construction Framework when: starting a new notebook or experiment; restructuring a disorganized notebook by discarding its existing structure and rebuilding section organization from scratch; or creating a reusable notebook template for a team or project.

Use the Audit Framework when: reviewing a collaborator’s notebook; preparing a notebook for production deployment or academic publication; conducting a pre-merge or pre-deployment review; or using an AI assistant to analyze submitted notebook code.

Use Both Frameworks iteratively when: authoring and self-reviewing during active development; collaborating or pair-programming in real time; or refactoring an existing notebook into a cleaner, production-ready version while preserving its existing section structure and logic.

The Construction Framework and Audit Framework complement each other within a feedback loop: the construction phase produces a notebook artifact, the audit phase diagnoses its weaknesses, and the resulting recommendations feed back into a revised construction cycle. This closure applies to notebooks that begin in the Building Phase; for notebooks entering via the audit-only path, the loop remains open unless the notebook is subsequently taken up for construction, as illustrated in Figure 1.